ACCESCO

CDAO Infrastructure, Replenishment Engines & Scalability Architecture

Consolidated technical report of the decisions, architecture, requirements, implementation plan and scalability recommendations discussed in this conversation

---

Scope

Customer-facing replenishment • Warehouse-facing replenishment • Enterprise data platform • Application architecture • One-million-user capacity planning • Deployment, testing and cost controls

Prepared as a consolidated planning document

Date: 7 November 2026

Document purpose and interpretation

This report consolidates the work discussed in the conversation into one coherent reference. It captures the selected direction, constraints, proposed technologies, architecture, data requirements, known gaps, deployment sequence and capacity-testing approach. It is intended to guide planning and technical implementation; it is not evidence that the infrastructure has already been provisioned, load-tested or approved for production.

Important distinction: several values in this report (especially concurrent-user and requests-per-second targets) are planning assumptions proposed for testing. They are not measured Accesco traffic, verified service limits or performance guarantees. Final sizing requires application profiling, workload modelling and load tests in the intended Azure region.

Contents

1. Executive summary
2. Business scope and constraints
3. Customer-facing replenishment engine
4. Warehouse-facing replenishment engine
5. Data requirements and preparation
6. Enterprise CDAO technology stack
7. End-to-end architecture and data flows
8. Scalability target: one million registered users
9. Deployment and implementation roadmap
10. Load-testing and release criteria
11. Security, governance and reliability
12. Cost management
13. Gaps, risks and decisions still required
14. Final recommended standard

1. Executive summary

The recommended direction is to standardise Accesco's data and analytics function on Microsoft Azure and Azure Databricks, while retaining PostgreSQL as the operational source of truth. ADLS Gen2 and Delta Lake provide the analytical storage foundation; Databricks provides ingestion, transformation, orchestration and machine-learning workflows; Power BI provides governed business reporting.

The replenishment capability consists of two distinct engines:

- **Customer-facing replenishment:** estimate when a household/customer is likely to run out of a product, show an expected depletion date or remaining duration, and send an opt-in reminder before depletion. The proposed model is XGBoost Survival AFT, supported by product-level/global consumption information for cold-start customers and a gradual shift toward personal purchase history.
- **Warehouse-facing replenishment:** forecast demand and translate forecasts into stock, transfer and supplier-order recommendations. The process must account for both internal inventory flows and external supplier replenishment, and staff approval is required before recommendations become actions.

The broader CDAO platform must also support enterprise reporting, customer and product analytics, supply-chain analytics, model governance and future AI use cases. The customer application and operational systems must scale independently from analytical workloads.

Core architectural principle: operational applications transact; the CDAO platform analyses and recommends. Model outputs should be published through controlled interfaces and must not directly overwrite operational records or bypass warehouse approval.

2. Business scope and constraints

2.1 Customer-facing requirements

- Primary purchase channel is the Accesco app, but the design should allow customers to record purchases made from offline shops or other retailers.
- Track product ID, product name, purchase date, quantity/size, price, and elapsed time since the previous purchase of the same product where available.
- Estimate consumption/depletion using both general product consumption information and personal purchase behaviour.
- New customers must be handled explicitly; the system should not pretend to have a personal history where none exists.
- Use a transition strategy: initially blend global/product-level patterns with personal evidence; after approximately 3–5 purchases, give greater weight to the customer's own observed history, subject to data quality and model validation.
- Notify customers before likely depletion, with a proposed initial target of two days before the predicted depletion date.
- Show likely depletion date/duration and relevant offers or discounts when available and valid.
- Customers should be able to enable/disable WhatsApp notifications; app push notifications are also required.
- Allow linked household/family accounts under a parent/main account so purchases and replenishment information can be shared according to permissions.

2.2 Warehouse-facing requirements

- Inventory movement includes both internal flows (warehouse/store transfers) and external replenishment (supplier orders).
- Monitoring and recommendation cycle: daily.

- Recommendations should be comprehensive, including what to replenish, quantity, urgency, and relevant operational constraints.
- Product catalogue and SKU identifiers are available.
- Warehouse staff approval is required before recommendations are executed.
- Product range is mixed, so the solution must accommodate different demand patterns, shelf lives, storage constraints and replenishment rules.

2.3 Current starting constraints

Area	Known position / constraint
Product master	SKU identifiers and product catalogue are available.
Customer history	No customer accounts or historical order records were available at the time of planning.
Warehouse history	Historical transaction and inventory availability must be audited and confirmed.
Location	Operating location has been finalised, but source data and integrations still need confirmation.
Application platform	Web/app backend and database technology had not been finalised in the earlier planning context.
Notifications	Push notification required; WhatsApp should be optional and consent-based.
Warehouse action	Staff approval required.
Cloud/data platform	Azure Data Lake and Databricks are the selected strategic direction; PostgreSQL is retained as the operational database recommendation.

3. Customer-facing replenishment engine

3.1 Objective and output

The engine estimates the time until a customer is likely to deplete a product. It should produce an expected depletion date or time-to-depletion estimate, an uncertainty/confidence representation, and a reminder decision. The customer experience should also show an estimated duration for a proposed purchase quantity, where the model has sufficient evidence.

A simple proportional rule (for example, 2 kg lasts twice as long as 1 kg) may be useful as an initial explanatory example, but it should not be treated as a universal law. Consumption can be nonlinear and influenced by household size, usage frequency, product type, seasonality, wastage and purchase context. Quantity scaling should be learned and validated.

3.2 Recommended model: XGBoost Survival AFT

Use XGBoost's Accelerated Failure Time (AFT) survival objective for the customer-side depletion-time problem. Survival modelling is appropriate because the true depletion time is often not directly observed. A customer may buy again before the old product is confirmed empty, may not report depletion, or may stop purchasing for reasons unrelated to consumption. These cases must not all be treated as exact depletion labels.

The AFT model estimates a time-to-event distribution rather than only a single point prediction. This supports an estimated depletion time and uncertainty intervals, which can be used to make reminder timing more responsible. The model is a recommendation for the stated problem, not a guarantee of accuracy before sufficient representative data are collected.

3.3 Event and censoring design

- **Observed event:** depletion is explicitly confirmed, for example through a customer feedback action such as “finished” or a sufficiently reliable observation.

- **Proxy event:** a repeat purchase can be used as an imperfect signal, but it does not prove that the previous quantity was fully consumed.
- **Right-censored observation:** the customer has not yet reported depletion by the end of the observation window; the product may still be in use.
- **Interval-censored observation:** depletion is known to have occurred between two observations, but the exact time is unknown.
- **Competing explanations:** a repeat purchase may reflect a promotion, bulk buying, a change in household demand, purchase from another retailer or stockpiling.

The training dataset must preserve the distinction between exact events, proxies and censored observations. Treating every reorder date as an exact depletion date can bias the model.

3.4 Cold start and personalisation

At the beginning, customer-specific history will be sparse or absent. The engine should use product-level/global priors or patterns, product attributes and optional customer-provided usage information. As reliable personal observations accumulate, personal features can contribute more strongly. The proposed 3–5 purchase transition is a starting business rule that must be validated empirically; it is not a universal statistical threshold.

A customer with insufficient history should receive a clearly labelled estimate with conservative uncertainty, or be asked for a small amount of useful information. Do not send high-confidence reminders based on weak evidence.

3.5 Customer-facing outputs and decision rules

- Predicted time-to-depletion and likely depletion date.
- Prediction interval or uncertainty band, plus a confidence/eligibility indicator.
- Estimated duration for the selected purchase quantity, only where supported by model evidence.
- Reminder eligibility and timing, initially targeting two days before likely depletion.
- Offer/discount information only after checking current product availability, offer validity, price and eligibility.
- Suppress or postpone reminders after a recent purchase, an explicit “still plenty” response, an opt-out, or a product availability issue.
- Track notification state: eligible, queued, sent, delivered where available, clicked, dismissed, purchased-after-notification, opted out and complained.

3.6 Customer feedback and external purchases

Because purchases may happen outside Accesco, the product experience should allow a customer to record an external purchase and optionally report remaining quantity or depletion status. Useful feedback actions include “Still plenty”, “Running low” and “Finished”. These actions create observations that improve future estimates. The system should record source/channel and confidence rather than treating an unverified customer entry as identical to a fulfilled Accesco order.

3.7 Household accounts

Linked accounts should be represented as a household or parent-account relationship with explicit permissions. The data model must define who can see a purchase, whether a household purchase updates a shared product estimate, how duplicate purchases are reconciled, and how a member can leave or revoke sharing. Avoid assuming that all household members have identical consumption patterns.

3.8 Customer engine evaluation

- Use chronological train/validation/test splits to prevent future information leaking into training.
- Evaluate survival likelihood and time-prediction quality using suitable survival metrics, such as concordance and survival-probability error measures, alongside calibration checks.

- Measure depletion-date error where reliable observed depletion labels exist.
- Measure reminder utility: reminders sent before depletion, false/early reminders, missed reminders, customer dismissals and opt-outs.
- Evaluate separately for new customers, repeat customers, product categories, pack sizes and household configurations.
- Compare against simple product-level and personal-history baselines before approving model deployment.

4. Warehouse-facing replenishment engine

4.1 Objective and recommended design

The warehouse engine should be designed as two connected components: (1) demand forecasting and (2) an inventory/replenishment planner. A forecast predicts future demand; the planner converts that forecast into actionable quantities while considering inventory position, lead times, service targets and operational constraints. A good forecast does not automatically guarantee a good replenishment decision.

4.2 Forecasting approach

Use a forecasting model selected through backtesting on Accesco's actual demand history. The earlier discussion considered TimesFM-3 as a possible foundation model for time-series forecasting. It can be evaluated as a candidate, but it should not be assumed to be the final production model until its licence, supported deployment route, data requirements, forecast quality and commercial-use conditions are confirmed. Databricks can be the main data/engineering platform while model inference is hosted in a compatible environment if necessary.

The warehouse forecasting design should support SKU-by-location demand series and account for intermittent demand, seasonality, promotions, stockouts and product lifecycle. Use simple baselines as a minimum evaluation benchmark. A short history such as one or two months may support an initial pilot for some high-frequency products, but it is generally insufficient to capture annual seasonality and many low-frequency SKUs.

4.3 Inventory planner inputs

- On-hand sellable stock and inventory status.
- Reserved, allocated, damaged, blocked and quarantined quantities.
- Open purchase orders and expected inbound receipts.
- Inter-location transfer quantities and in-transit status.
- Supplier lead times, delivery calendars, minimum order quantities and pack sizes.
- Safety-stock/service-level policy and target cover days.
- Storage capacity, shelf life, batch/expiry data and handling constraints.
- Budget, supplier limits, transport capacity and operational priorities.

4.4 Recommendation flow

- Forecast expected demand over the relevant replenishment horizon.
- Calculate inventory position using available stock, reservations, inbound supply and committed demand.
- Identify projected stockout dates and service-level risk.
- Check whether a transfer from another eligible node is feasible and preferable.
- Otherwise calculate a supplier replenishment recommendation subject to lead time, MOQ, pack rounding and capacity.
- Apply expiry, shelf-life and sell-through checks for perishable products.

- Flag infeasible or conflicting constraints rather than silently generating an arbitrary order.
- Present the recommendation, rationale, forecast, assumptions and risk to warehouse staff for approval.
- Record approval, rejection, quantity override and reason; track the outcome for later evaluation.

4.5 Internal allocation and external replenishment

The engine must distinguish stock movement inside Accesco's network from purchasing stock from suppliers. A recommended decision sequence is: evaluate available eligible stock at source nodes; determine whether a transfer can arrive in time and within transport/capacity constraints; then calculate external replenishment for any remaining shortage. Allocation priorities should consider urgency, expected demand, service targets, transport constraints and fairness across locations. This sequence requires business approval and may differ by product or location.

4.6 Warehouse evaluation

- Forecast accuracy and bias by SKU, location and demand pattern.
- Stockout-aware evaluation: periods when an item was unavailable must not automatically be interpreted as zero demand.
- Inventory outcomes: service level, stockout frequency/duration, excess stock, expiry/waste and inventory turns.
- Planner outcomes: recommendation acceptance, overrides, supplier order accuracy and transfer effectiveness.
- Backtest the forecasting model separately from the complete inventory planning loop.
- Run shadow recommendations before staff rely on them operationally.

5. Data requirements and preparation

5.1 Customer engine data dictionary

Data group	Required attributes / examples	Purpose
Customer/account	customer_id, household_id, account_status, consent_preferences, created_at	Identity, household sharing and communication eligibility
Product catalogue	sku_id, product_name, category, brand, unit_of_measure, pack_size, net_quantity, product_status	Product identity and quantity normalisation
Purchase/order line	order_id, customer_id, sku_id, purchase_datetime, quantity, unit, unit_price, channel, fulfilment_status	Purchase history and customer-product timeline
External purchase	customer_id, sku_id, purchase_datetime, quantity, source/channel, user-entered flag	Capture purchases outside Accesco
Repeat interval	previous_purchase_datetime, elapsed_days, previous_quantity	Derived behavioural features; calculate from event history
Depletion/feedback	event_type, observed_at, remaining_quantity, event_source, confidence	Observed, proxy and censored event construction
Context	household_size if volunteered, season, promotion, product attributes	Explain variation in consumption
Notification	channel, consent, eligible_at, sent_at, delivery_status, click/dismiss/opt-out	Policy, delivery and outcome measurement
Offer/availability	offer_id, sku_id, start/end, discount, stock_status, eligibility	Ensure displayed offers are valid

5.2 Warehouse engine data dictionary

Data group	Required attributes / examples	Purpose
Product/SKU	sku_id, category, unit, pack_size, shelf_life, storage constraints	Forecast and order unit standardisation
Location	location_id, node_type, capacity, operating calendar	SKU-location forecasting and network planning
Demand history	date/time, sku_id, location_id, fulfilled_qty, ordered_qty, cancelled_qty	Demand series and reconciliation
Availability/stockout	on_hand, available_qty, stockout_start/end, unavailable flag	Avoid interpreting constrained sales as true demand
Inventory snapshot	on_hand, reserved, allocated, damaged, blocked, quarantined	Inventory position
Inbound supply	PO_id, supplier_id, order_qty, expected_date, received_qty, status	Incoming stock and lead-time analysis
Transfers	source_node, destination_node, sku_id, qty, dispatch/arrival time, status	Internal network replenishment
Supplier	supplier_id, lead_time history, MOQ, pack multiple, delivery calendar	Supplier ordering constraints
Cost/expiry	unit_cost, batch_id, expiry_date, storage capacity	Cost, waste and perishability decisions
Planner outcomes	recommendation, approved qty, override reason, actual receipt, stockout/waste outcome	Business evaluation and feedback

5.3 Shared data engineering requirements

- Canonical SKU, customer, household, location, supplier and order identifiers.
- Source-system ownership for each field and documented data lineage.
- Consistent time zones, event timestamps, units of measure and status definitions.
- Incremental ingestion with source timestamps, load IDs and reconciliation counts.
- Data-quality checks for duplicates, null identifiers, impossible quantities, invalid dates and inconsistent status transitions.
- Historical snapshots and retention policies so model training can reproduce the data available at a past point in time.
- Quarantine/reject handling and alerts for stale or conflicting data.
- Access controls and privacy safeguards for customer-level data.

5.4 Minimum data before training

For the customer engine, first collect customer-linked purchase events and feedback; a catalogue alone cannot train a personalised depletion model. Start with product-level priors and a controlled pilot while building history. For the warehouse engine, verify historical fulfilled demand, stock availability/stockout periods, inventory snapshots, inbound purchase orders, transfers and lead times. Without availability data, sales history can understate true demand during stockouts.

6. Enterprise CDAO technology stack

The following is the consolidated target stack for the whole CDAO function, not only replenishment. It is designed to provide one governed analytical foundation for enterprise BI, supply-chain analytics, customer analytics, product analytics and future predictive/AI projects.

Layer	Recommended technology	Role
Cloud	Microsoft Azure	Shared cloud environment
Operational source	PostgreSQL and approved system APIs	Transactional source of truth
Web frontend	Next.js, React, TypeScript	Customer/admin web applications
Mobile frontend	React Native if mobile app is required	Mobile customer experience
Backend/API	Existing backend services; Python/FastAPI for suitable new services and model APIs	Business logic and serving
Edge/security	Azure Front Door + WAF	Routing, edge delivery and web protection
Application hosting	Azure Container Apps initially; App Service alternative	Managed scalable application compute
Packaging	Docker	Consistent deployment packaging
Operational database	Azure Database for PostgreSQL Flexible Server	Users, orders, products, inventory and transactions
Connection management	PgBouncer / connection pooling	Control application-to-database connections
Cache	Managed Redis-compatible cache where justified	Frequently read data and suitable cache workloads
Messaging	Azure Service Bus	Durable asynchronous work and integration
Lake storage	ADLS Gen2	Raw and historical analytical storage
Lakehouse	Azure Databricks + Delta Lake	Central analytics platform
Processing	PySpark + SQL	Large-scale data transformations
Transformation	dbt	Reusable tested analytical models
Orchestration	Databricks Workflows	Scheduled pipelines and dependencies
BI	Databricks SQL + Power BI	Governed reporting and dashboards
ML lifecycle	Databricks ML + MLflow	Experiments, model registry/lifecycle and evaluation
Governance	Unity Catalog	Data discovery, lineage and access governance
Identity/secrets	Microsoft Entra ID + Azure Key Vault	Identity, access and secrets
CI/CD	GitHub + GitHub Actions	Version control and automated deployment
Infrastructure as code	Terraform when environment automation is mature	Repeatable provisioning
Monitoring	Azure Monitor + Application Insights	Metrics, logs, tracing and alerts
Load testing	Azure Load Testing	Concurrent traffic and performance tests
Documentation	Confluence + GitHub	SOPs and technical records

6.1 Technology selection principles

- Use the current operational source of truth; do not create a competing inventory or order ledger in the CDAO platform.
- Use the shared lakehouse and governance layer across projects instead of independent data silos.
- Start with scheduled batch/incremental ingestion; add streaming only when a measured latency requirement needs it.
- Use Databricks Workflows initially rather than introducing multiple overlapping orchestrators.

- Do not require Kafka, Kubernetes/AKS, database sharding or a separate enterprise data warehouse on day one. Introduce only when justified by measured requirements.
- Keep model training/scoring and reporting isolated from live transactional request paths.

7. End-to-end architecture and data flows

7.1 Operational and analytical separation

Operational applications handle customer and staff transactions. PostgreSQL and the relevant operational systems remain authoritative. The CDAO platform receives approved extracts/events, preserves source evidence and creates curated analytical data. It returns recommendations or predictions through controlled outputs and APIs.

7.2 Data lakehouse layers

Layer	Contents	Controls
Bronze	Raw source extracts, API events, source snapshots	Preserve source values, load timestamps, source IDs and ingestion metadata
Silver	Validated, deduplicated, standardised and joined data	Schema checks, business rules, unit/time normalisation, quarantine
Gold	Business-ready facts, dimensions, KPIs, features and model outputs	Governed definitions, access controls, lineage and quality thresholds

7.3 Replenishment data flow

- Operational systems publish or expose orders, products, inventory, transfers, suppliers, customer events and fulfilment records.
- Databricks Workflows ingests incrementally into ADLS Gen2/Delta Bronze.
- PySpark/SQL and dbt validate and standardise the data into Silver.
- Gold tables provide customer-product event histories, SKU-location demand series, inventory positions and trusted business metrics.
- Customer survival model and warehouse forecasting/planner jobs run on scheduled or event-triggered workflows according to the approved freshness requirement.
- Model outputs are versioned and written to controlled serving/output tables or APIs.
- Customer notification services and warehouse recommendation screens consume those outputs through business rules and permissions.
- Power BI consumes Gold datasets for monitoring, reporting and management decisions.

7.4 Model output governance

Every production prediction should carry the model/version identifier, prediction timestamp, input-data freshness, relevant horizon, uncertainty/quality indicator and decision status. Warehouse recommendations additionally need rationale, constraints, approval status, approver and override reason. Customer notifications must record consent and notification outcomes.

8. Scalability target: one million registered users

8.1 Registered users versus concurrent users

One million registered accounts do not mean one million simultaneous sessions. Capacity is driven by active users, requests per second, request mix, transaction rate, database work, external dependencies and peak patterns. The planning assumption used in the discussion was a peak of 10,000–20,000 concurrent active users, not one million concurrent users.

Planning scenario	Concurrent active users	Indicative API throughput target
Initial launch	100–500	100–300 requests/sec
Growth	1,000–5,000	500–2,000 requests/sec
High-traffic peak	10,000–20,000	2,000–5,000 requests/sec
Stress exploration	30,000+	5,000+ requests/sec

These are proposed load-test scenarios only. They are not forecasts, vendor capacity guarantees or proof that a particular configuration can sustain the load. The request rate must be recalculated from Accesco's expected user journeys, requests per session, session duration, checkout rate and peak concentration.

8.2 Application architecture additions

- Azure Front Door with WAF for edge routing, content delivery and protection.
- Stateless, horizontally scalable API services hosted initially on Azure Container Apps or App Service.
- Azure Database for PostgreSQL Flexible Server with production high availability, backups and point-in-time recovery.
- Connection pooling to prevent excessive concurrent application connections.
- Managed cache for suitable catalogue, configuration and other frequently read data; cache invalidation and correctness must be designed.
- Azure Service Bus for asynchronous notifications, background tasks and retryable integration work.
- Application Insights and Azure Monitor for end-to-end request and dependency observability.
- Separate scaling and capacity controls for application compute, database, cache, queues and Databricks.

8.3 What not to deploy prematurely

Do not automatically introduce AKS, Kafka, database sharding, multiple microservices or a separate enterprise warehouse simply because the registered-user target is one million. These components add cost and operational complexity. Add them when profiling demonstrates a need, such as a requirement for orchestration flexibility, high-volume event streaming, workload isolation or database scale beyond a well-tuned managed PostgreSQL deployment.

8.4 Important limitation

The selected CDAO stack alone does not guarantee the application can serve one million users. Application/backend design, database schema and queries, connection limits, caching, external services, network architecture and tested deployment configuration determine end-user capacity. A one-million-concurrent-user requirement would be a substantially different target and would require a separate architecture and capacity assessment.

9. Deployment and implementation roadmap

Phase	Work to complete	Exit condition
1. Foundation	Azure subscriptions/landing zone, identity, networking, environments, Key Vault, GitHub, monitoring, cost budgets	Secure dev/test/prod foundations and owners approved
2. Application baseline	API hosting, Front Door/WAF, PostgreSQL, pooling, backups, queue, cache where justified	Core user journeys work with reliable data and recovery
3. CDAO foundation	ADLS Gen2, Databricks, Delta, ingestion, Bronze/Silver/Gold, Unity Catalog	Trusted source-to-Gold pipelines and data-quality checks
4. BI and data products	Metric definitions, Gold datasets, Databricks SQL, Power BI	Business owners validate KPI definitions and dashboards
5. Model development	Customer event labels, warehouse demand data, baselines, model evaluation, MLflow	Model evidence and acceptance criteria approved
6. Shadow/pilot	Predictions/recommendations run without uncontrolled actions; staff review warehouse recommendations	Pilot thresholds met and issues resolved
7. Capacity validation	Load, stress, spike, endurance, failover and restore tests	Measured capacity, recovery and cost evidence
8. Controlled production	Limited rollout, feature flags, alerts, rollback and operational support	Go-live approval and runbooks in place
9. Scale and optimise	Tune from real traffic, add capacity or advanced components where justified	SLOs and budget maintained as demand grows

Recommended environment separation: development, staging/test and production. Use separate credentials, access policies, data handling rules and deployment approvals. Production data should not be copied into development without an approved masking or privacy process.

10. Load-testing and release criteria

Use Azure Load Testing or an equivalent load-generation tool to exercise complete user journeys against a production-like staging environment. Test API, database, cache, queues and external dependencies together.

Test	Proposed target	Initial pass criteria
Baseline	100–1,000 concurrent users	Establish throughput, p50/p95/p99 latency, error rate and resource use
Growth	5,000 concurrent users	Ordinary read API p95 target under 500 ms; agreed write targets met
Peak	10,000 concurrent users	Ordinary read API p95 target under 500 ms; critical writes p95 under 1 sec
Higher peak	20,000 concurrent users	No data corruption; agreed latency/error budgets maintained
Spike	Short 5× traffic increase	Autoscaling, rate limiting and queueing avoid cascading failure
Endurance	8–24 hours at representative load	No sustained memory growth, connection exhaustion or degradation
Failure/recovery	Database failover or instance loss	RTO/RPO and service recovery targets met
Security	Authenticated/unauthenticated abuse scenarios	WAF, rate limits, authorization and access controls work

These latency values are proposed starting service-level objectives, not universal guarantees. Define separate targets for catalogue browsing, search, checkout, payment integration, order tracking and internal staff operations. Payment-provider latency should be measured separately from Accesco's own API processing.

10.1 Required measurements

- Requests per second, concurrent sessions and transaction rate.
- Latency percentiles (p50, p95 and p99), timeout rate and HTTP error rate.
- Application CPU, memory, instance count, restart count and autoscaling delay.
- PostgreSQL CPU, memory, I/O, active connections, pool wait time, slow queries and replication lag.
- Cache hit ratio, evictions and latency.
- Service Bus queue depth, message age, dead-letter count and processing rate.
- External dependency latency and failure rate.
- Data correctness: duplicate orders, inventory inconsistencies, lost events and retry idempotency.

10.2 User-journey mix

Simulate realistic browsing, search, product detail, cart changes, checkout, payment-provider interaction, order creation, inventory checks and order tracking. A test consisting only of health checks or cached reads will not demonstrate checkout capacity. Seed representative product, customer, order and inventory data before testing.

11. Security, governance and reliability

11.1 Security and privacy

- Use Microsoft Entra ID for workforce identity and managed identity/service principals for workloads where supported.
- Apply least privilege through Azure RBAC, Databricks permissions and Unity Catalog.
- Store credentials and keys in Azure Key Vault; rotate secrets and avoid embedding them in notebooks or code.
- Encrypt data in transit and at rest; restrict database and storage network access.
- Apply customer-level authorization on every relevant API; do not rely only on frontend filtering.
- Define household sharing permissions, customer consent, notification preferences and data deletion/retention rules.
- Mask or anonymise personal data in non-production environments.
- Maintain audit records for sensitive access, data changes, model releases and warehouse approvals.

11.2 Reliability and recovery

- Agree RTO (how quickly service must recover) and RPO (how much data loss is acceptable) for each critical workload.
- Configure database backups and point-in-time recovery; periodically perform restoration drills.
- Use retries with backoff, idempotency keys and dead-letter handling for asynchronous work.
- Monitor source-data freshness and prevent stale or conflicting inventory data from producing unsafe recommendations.
- Use deployment health checks, staged rollout and rollback procedures.
- Document operational ownership, on-call escalation and incident response.

11.3 Model governance

Track training data version, feature definitions, code version, model parameters, evaluation results, approval, deployed model version, scoring time and observed outcomes. Retraining should be triggered by an approved schedule or evidence of drift/quality degradation, not simply because a new date has arrived.

12. Cost management

The cost strategy is to provision for measured current demand, use elastic capacity for peaks, and avoid keeping expensive analytics compute running when idle. A reliable monthly estimate cannot be calculated from registered-user count alone. It requires region, service tiers, database size, request volume, data volume, retention, concurrency, availability and networking assumptions.

Cost area	Control strategy
Application compute	Autoscale within tested min/max limits; scale in when demand falls; track cost per API request
PostgreSQL	Choose a tested baseline; monitor CPU, I/O, storage and connections; resize based on evidence; retain required HA
Cache	Cache only workloads with measurable benefit; monitor hit rate, memory and eviction
Service Bus	Monitor message volume, queue age, retries and dead-letter rates
Databricks	Schedule jobs; use job-appropriate compute; configure autoscaling/auto-termination where suitable; avoid always-on clusters
ADLS/Delta	Set retention and lifecycle policies; compact/optimise tables where justified; avoid unnecessary duplicate copies
Power BI	Use curated Gold datasets and appropriate refresh modes; avoid repeated heavy source queries
Monitoring/logging	Set retention, filter noisy telemetry and track ingestion costs
Networking	Monitor egress, cross-region transfers and private connectivity costs
Environments	Keep dev/test smaller; stop idle compute; use budgets and resource tags

Set budget alerts at 50%, 75%, 90% and 100% of approved monthly budgets. Alerts do not necessarily stop spending; enforce controls through quotas, scaling limits, resource policies and operational procedures. Track cost by environment, workload, owner and cost centre.

13. Gaps, risks and decisions still required

The architecture is a target design. The following issues must be resolved before implementation can be considered production-ready.

13.1 Customer engine gaps

- Define how reliable depletion labels will be collected and distinguish confirmed depletion from repeat-purchase proxies.
- Finalise cold-start behaviour, confidence thresholds and the empirical rule for shifting toward personal data after 3–5 purchases.
- Specify quantity-duration learning, pack-size normalisation and non-linear consumption effects.
- Define household data sharing, duplicate purchase reconciliation and account unlinking.
- Define external-purchase verification and feedback quality.
- Specify reminder suppression, cooldown, quiet hours, offer validation, product unavailability and notification failure handling.
- Define privacy, consent, retention and WhatsApp provider integration.

13.2 Warehouse engine gaps

- Confirm authoritative inventory, order, transfer and supplier systems and their APIs.
- Define central warehouse/store allocation priorities and the transfer-versus-supplier decision sequence.

- Define stock freshness and reconciliation thresholds; stale or conflicting inventory should block affected recommendations.
- Model supplier and transport lead-time uncertainty.
- Define batch/expiry and perishable-stock allocation rules.
- Resolve conflicts among MOQ, pack rounding, budget, storage capacity, supplier limits and transport constraints.
- Approve measurable service-level, stockout, excess-stock, waste and staff-workload targets.

13.3 Enterprise-wide decisions

- Confirm Azure region, subscription/landing-zone design, network topology and data residency requirements.
- Confirm operational database schema, owners, data volumes, retention and expected growth.
- Confirm expected active users, peak concurrent sessions, requests per session, checkout rate and promotional traffic profile.
- Agree availability targets, RTO/RPO, support ownership and incident escalation.
- Confirm service availability and licensing/terms for every selected managed service and model.
- Define monthly budget, cost allocation and capacity approval process.
- Decide whether any real-time/streaming use case is required; do not assume daily batch is insufficient without a latency requirement.

13.4 TimesFM-3 and commercial-use note

TimesFM-3 was discussed as a possible warehouse forecasting model and the possibility of using Google Cloud/BigQuery ML was raised. The final enterprise recommendation in this report keeps Azure and Databricks as the primary platform. Do not assume that paying for Google Cloud or BigQuery ML automatically grants unrestricted commercial rights to a model or that every model version is available through BigQuery ML. Verify the exact model version, licence, hosting/inference availability, terms and cost before commercial deployment. A model can be developed or evaluated outside the main lakehouse and still be integrated into a Databricks-centred data architecture.

13.5 Implementation readiness

The previous documentation audit found the customer and warehouse guides to be strong planning documents, but not fully implementation-ready. The main improvements were to make customer cold-start/household/feedback rules, warehouse network allocation/perishability/stock reconciliation rules, shared data ownership and measurable acceptance criteria more explicit. The audit was a documentation review, not a validation that the systems or integrations already exist.

14. Final recommended CDAO standard

The recommended enterprise standard is:

Domain	Final standard
Cloud	Microsoft Azure
Application edge	Azure Front Door + WAF
Application hosting	Azure Container Apps initially or Azure App Service
Frontend	Next.js / React / TypeScript; React Native if mobile is required
Backend	Existing application backend; Python/FastAPI for suitable services and ML APIs
Operational database	Azure Database for PostgreSQL Flexible Server

Domain	Final standard
Database connection management	PgBouncer / connection pooling
Cache and messaging	Managed Redis-compatible cache where justified; Azure Service Bus
Analytical storage	ADLS Gen2
Lakehouse and compute	Azure Databricks + Delta Lake
Data engineering	PySpark + SQL + dbt + Databricks Workflows
BI	Databricks SQL + Power BI
ML lifecycle	Databricks ML + MLflow
Governance	Unity Catalog + Entra ID + Azure Key Vault
CI/CD and IaC	GitHub Actions; Terraform when provisioning automation is ready
Monitoring/testing	Azure Monitor + Application Insights + Azure Load Testing
Documentation	Confluence + GitHub

Final architecture principles

- PostgreSQL and operational applications remain the source of truth for transactions.
- Azure Databricks and ADLS Gen2 form the shared CDAO analytical foundation.
- Customer and warehouse replenishment remain separate decision engines but share governed product, location, customer and operational data where appropriate.
- Keep transactional workloads isolated from analytical processing and model training.
- Begin with manageable, cost-controlled capacity; scale after realistic load tests and observed demand.
- Require data quality, security, model governance, staff approval and rollback controls before production use.
- Treat one million registered users and 10,000–20,000 concurrent users as a planning scenario until validated with Accesco-specific traffic assumptions and load tests.

Conclusion

Accesco should build one enterprise data platform rather than a collection of disconnected analytics projects. Azure and Databricks provide the shared foundation; PostgreSQL remains the operational source of truth; Power BI provides governed reporting; and MLflow supports controlled predictive-model development. The two replenishment engines can then be implemented as governed data products on top of this foundation. The application tier must be designed and tested separately for high concurrency. The correct strategy is to design for growth, launch with measured capacity, and expand infrastructure only when performance, reliability or business requirements demonstrate the need.

End of report